

What is Parallel and Distributed Computing

- **Parallel and Distributed Computing** are two closely related concepts in computer science that focus on improving performance, efficiency, and scalability by using multiple computing resources at the same time.
- **Parallel computing** is a type of computation where **multiple processing units work simultaneously on different parts of the same problem**.

Key Idea

- *Do many calculations at the same time within one system.*

Parallel Computing

- **Parallel computing** is a type of computation where **multiple processing units work simultaneously on different parts of the same problem**.

Key Idea

- *Do many calculations at the same time within one system.*

Characteristics of Parallel Computing

- Uses **multiple CPUs or cores** in a single computer
- Tasks are divided into **smaller subtasks**
- Subtasks run **simultaneously**
- High-speed communication (shared memory)

Example

- A multicore processor running different parts of a matrix calculation at the same time
- GPU processing thousands of pixels simultaneously in image processing

Common Technologies

- Multicore processors
- GPUs
- OpenMP, CUDA, MPI (in shared-memory systems)

Simple Example

If a program needs to process **1,000,000 numbers**, it can:

- Split the data into 4 parts
- Assign each part to a different CPU core
- Process them at the same time

Distributed Computing

Distributed computing is a system where **multiple independent computers (nodes) work together over a network** to solve a problem.

Key Idea

- *Many computers cooperate to solve a task.*

Characteristics of Distributed Computing

- Uses multiple computers
- Each computer has its own memory
- Communication happens over a network
- More scalable and fault-tolerant

Example

- Cloud computing systems (e.g., Google Cloud, AWS)
- A web application running on multiple servers
- Distributed databases like Cassandra or MongoDB

Common Technologies

- Hadoop
- Spark
- Distributed systems using TCP/IP
- Microservices architecture

Simple Example

A search engine:

- One server stores web pages
- Another processes search queries
- Others rank and return results

Parallel vs Distributed Computing (Comparison)

Relationship between Parallel and Distributed Computing

- **Distributed systems can use parallel computing internally**
- Modern systems (e.g., cloud data centers) often combine **both**
- Parallel computing within each server
- Distributed computing across many servers

Why It Matters in Computer Science

- Handles **large-scale problems**
- Improves **performance and speed**
- Supports **big data, AI, scientific simulations, and cloud computing**

What is Parallelism?

- Parallelism is performing multiple tasks or parts of a task at the same time
- using multiple processors or CPU cores.
- Goal: Faster execution and better performance.

Example of Parallelism

Real-life Example:

- A restaurant kitchen where chefs cook different dishes at the same time.

Computer Example:

- A **multi-core processor** running multiple calculations at once. For example, when editing a video:
- One core processes video frames
- Another core applies effects
- Another core handles audio
- ****Simple programming example:****Processing a large dataset where different portions of the data are analyzed simultaneously using multiple CPU cores.
- 👉 In short, **parallelism = doing many things at the same time to finish faster**

Scenario: A computer with **4 CPU cores**

Time →

Core 1: [Task A]

Core 2: [Task B]

Core 3: [Task C]

Core 4: [Task D]

- ✓ All tasks run **at the same time**
- ✓ Requires **multiple processors or cores**

What is Concurrency?

- Concurrency is handling multiple tasks at once by switching between them.
- Tasks may not run simultaneously but appear to run at the same time.
- Goal: Responsiveness and efficient task management.

Example of Concurrency

Real-life Example:

- A teacher managing different student concerns one at a time.

Computer Example:

- A single-core CPU running and playing music, browsing, and download a file by task switching.
- ****Simple programming example:**** A web server handling multiple user requests by interleaving their execution rather than finishing one request before starting another.
- 👉 In short, **concurrency = managing multiple tasks at once**, while **parallelism = executing multiple tasks at the same time**.

Scenario: A computer with **1 CPU core**

Time →

CPU: [Task A] → [Task B] → [Task C] → [Task A] → [Task B]

- ✓ Only **one task runs at a time**
- ✓ CPU switches tasks very fast (**context switching**)
- ✓ Tasks appear to run simultaneously

Concurrency vs Parallelism

Concept

Illustration Idea

Core Idea

Parallelism	Multiple workers working together	Same time execution
Concurrency	One worker multitasking	Task switching

Parallel Computer Architectures

- **Parallel computer architecture** is a type of computer design where **multiple processors (or cores) work at the same time** to solve a problem. Instead of one processor doing all the work step-by-step, the task is **divided into smaller parts**, and each part is processed **simultaneously**.

Why Parallel Architecture

- Faster processing
- Can handle large and complex problems
- Better performance for data-intensive tasks

What is Parallel Architecture

- Uses multiple processors or CPU cores
 - Executes tasks simultaneously
 - Divides problems into smaller parts
 - Improves speed and performance

Key Idea (Simple)

- 👉 *One problem → split into many tasks → processed at the same time*

Types of Parallel Architecture

1. **SIMD (Single Instruction, Multiple Data)**
 1. One instruction is applied to **many data items at once**
 2. Used in graphics and scientific calculations
 3. **Examples:**
 - GPUs (Graphics Processing Units)
 - Vector processors
 - Image processing systems

Example: Adding 10 to **1,000 numbers at the same time**

1. MIMD (Multiple Instruction, Multiple Data)

1. Multiple processors run **different instructions** on **different data**
2. Most common today
3. **Examples:**
 - Multi-core CPUs (Intel Core i5/i7, AMD Ryzen)
 - Supercomputers
 - Distributed systems

Example:

One core runs a browser, another runs music, another runs a game

1. Shared Memory Architecture

1. All processors share the **same memory**
2. Easy to communicate, but may cause congestion
3. **Examples:**
 - Multi-core desktop computers
 - Servers with multiple CPUs

2. Distributed Memory Architecture

1. Each processor has **its own private memory**
2. Communication is done via network
3. **Examples:**
 - Computer clusters
 - Cloud computing systems
 - Supercomputers like **Beowulf clusters**

Real-World Examples of Parallel Computers

- **Modern PCs & laptops** (multi-core processors)
- **Supercomputers** (weather forecasting, simulations)
- **GPUs** (gaming, AI, machine learning)
- **Smartphones** (multiple cores for apps, camera, AI)